Cloud Provider Analytics, entrega 2

Proyecto integrador de Big Data, ITBA, segundo cuatrimestre 2026. Segunda entrega: implementación técnica y MVP end-to-end mínimo. Versión 1, 2026-11-08.

Grupo 11: Ivan Odzomek, Lucas Campoli, Matias Sapino, Diego Rabinovich, Diego Badin.

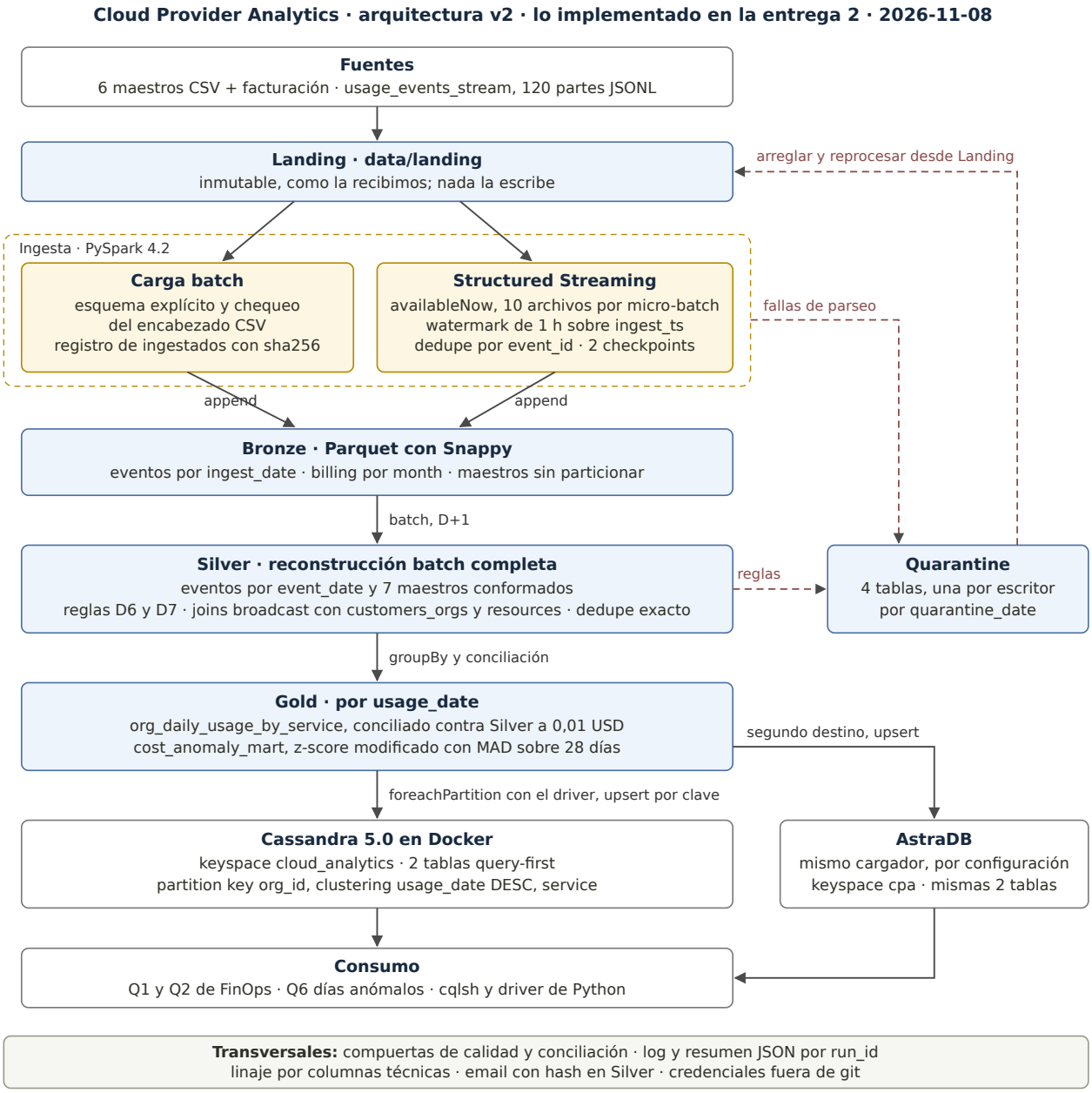
El pipeline corre de punta a punta sobre los datos provistos: Landing, Bronze, Silver, Gold y Cassandra, con batch y streaming, reglas de calidad con Quarantine, un mart FinOps, un componente de anomalías y evidencia de idempotencia. Este documento resume lo implementado y remite a la evidencia. Los pasos exactos para correrlo están en el `README.md`, las decisiones en `DECISIONS.md` y los reportes generados en `evidence/entrega-2/`.

1. Qué cambió desde la entrega 1

La entrega 1 no tuvo correcciones de la cátedra. Lo que cambió en el diseño viene de implementarlo:

Cambio	Por qué	Decisión
La vista intradía provisoria (speed layer) no está, y sale del diagrama	la entrega 2 pide el mart diario; queda en el backlog	D3
El streaming corre con `availableNow` y `processingTime="1 minute"` queda como modo continuo	el pipeline se corre entero, se re-ejecuta y se muestra en vivo	D14
Gold particiona por `usage_date`	es la fecha del evento con el nombre del grano del mart	D5
La carga a Cassandra usa el driver de Python desde `foreachPartition`	el conector oficial no tiene versión para Spark 4	D16
Quarantine son cuatro tablas, una por escritor, con `rule_name`	el sink del stream es dueño de su carpeta y las reglas se reconstruyen	D15
Nueva regla de rechazo `resource_org_mismatch`	un recurso de otra organización es una contradicción entre claves	D6
El dedupe del stream usa un hash del contenido cuando falta `event_id`	los nulos se agrupan entre sí y colapsarían filas distintas	D14
Registro de ingestados con sha256, dedupe por clave en Silver	idempotencia de la carga batch	D13
`users.email` se reemplaza por un hash con sal en Silver	es el único dato personal	D17
D11 y D12 cerradas con los conteos de Gold	estaban abiertas a propósito	D11, D12
Spark local en WSL, Cassandra en Docker	el supuesto de §8.1 del diseño preveía este cambio	D16

2. Arquitectura v2



La figura muestra solo lo que corre. El patrón sigue siendo un híbrido estilo Lambda (D3): el streaming hace append a Bronze y todo lo demás es batch, porque un watermark sobre el tiempo del evento descartaría casi todos los eventos (§4.1 del diseño). La vista intradía provisoria del speed layer no está implementada y por eso no aparece. El serving tiene dos destinos probados con el mismo cargador (D16): Cassandra 5.0 en Docker, que es el principal, y AstraDB, que se elige por configuración.

Etapa	Comando	Entrada	Salida
Carga batch	`make bronze`	7 CSV de Landing	`bronze/<maestro>`, `quarantine/masters_parse`
Streaming	`make stream`	`usage_events_stream/*.jsonl`	`bronze/usage_events`, `quarantine/usage_events_parse`
Silver	`make silver`	Bronze	`silver/*`, `quarantine/*_rules`
Gold	`make gold`	Silver	`gold/org_daily_usage_by_service`, `gold/cost_anomaly_mart`
Serving	`make serve` y `make queries`	Gold	2 tablas en Cassandra y en AstraDB, salidas de Q1, Q2 y Q6

`make run` corre todas las etapas en orden. Cada corrida tiene un `run_id`, escribe `logs/<run_id>.log` y un resumen JSON con la configuración, el commit y las métricas de cada etapa, y termina con código 1 si alguna etapa falla.

3. Procesamiento con Spark

3.1 Bronze

La carga batch lee cada CSV con su `StructType`, compara el encabezado con el esquema (`enforceSchema=false`) y manda a Quarantine, con la línea cruda, lo que no parsea. Agrega `ingest_ts`, `ingest_date` y `source_file` (desde `_metadata.file_name`). Billing particiona por `month` y los maestros quedan sin particionar (D5).

El streaming son dos queries sobre el mismo directorio, cada una con su `queryName` y su checkpoint. La de Bronze separa primero las fallas de parseo, después hace `withWatermark("ingest_ts", "1 hour")` y `dropDuplicatesWithinWatermark` sobre la clave de dedupe, y escribe un archivo por micro-batch. Sobre los datos reales:

Medida	Valor
micro-batches con datos	12, de 10 archivos y 3.600 eventos cada uno
eventos en Bronze	43.200
filas descartadas por el watermark	0
filas en el estado del dedupe al final	43.200 (todas llegaron dentro de la hora)
re-ejecución con el mismo checkpoint	0 filas leídas
experimento con watermark de 2 s	0 descartadas, estado máximo 24.480 claves, 24.480 liberadas

El experimento (`make stream-state-demo`) es la prueba empírica de D8: el estado se libera a medida que avanza el watermark y Bronze sigue recibiendo todos los eventos. La late data se mide contra el tiempo del evento: con un watermark de 1 día sobre `timestamp`, el 97,5% de los eventos (42.118) habría sido tardío. Bronze los conserva todos y Silver ubica cada uno en su `event_date` (`stream_bronze.md`).

3.2 Silver

Silver se reconstruye entero desde Bronze en cada corrida (D8). Los maestros se quedan con la última fila por clave natural y reciben las reparaciones de D7. Los eventos pasan por:

1. Conformación de `service`, `region`, `metric` y `unit` (minúsculas y sin espacios).
2. Dos joins broadcast, contra `customers_orgs` y `resources`, que agregan industria, plan, estado del recurso y dueño del recurso. Un join sin match o un recurso de otra organización son reglas de rechazo.
3. Las 8 reglas de rechazo, la imputación de `unit` y el `try_cast` de `value` (D2, D6).
4. Dedupe exacto por `event_id` con una ventana que conserva la primera fila por `ingest_ts`.
5. Features: `requests`, `cpu_hours` y `storage_gb_hours` como una columna por métrica, más `cost_usd_increment`, `genai_tokens` y `carbon_kg`.

Silver queda particionado por `event_date`, con 60 particiones y un archivo de unos 46 KB por día.

3.3 Gold y el plan de ejecución

`org_daily_usage_by_service` es un `groupBy(org_id, usage_date, service).agg(...)` con `daily_cost_usd`, `requests`, `cpu_hours`, `storage_gb_hours`, `genai_tokens`, `carbon_kg`, `events`, `negative_cost_events` y `unit_imputed_events`. `F.sum` devuelve null cuando todas las entradas son nulas, que es el par suma y conteo del §7 del diseño. Antes de escribir, el mart se concilia contra Silver: si alguna organización y día difiere en más de 0,01 USD, si hay claves nulas o si la cantidad de filas no es la del grano, la corrida falla y no publica.

Control	Resultado
filas en Gold	11.050
claves del grano en Silver	11.050
eventos sumados en Gold	43.200
organización-días comparados	4.580
diferencia máxima	0,00 USD

El plan físico (`gold_explain.txt`) es el job de MapReduce del §7:

Nodo del plan	Etapa de MapReduce
`Scan parquet` que lee 11 de las 28 columnas de Silver (poda de columnas)	input splits y lectura del map
`HashAggregate` con `partial_sum` y `partial_count`	combiner, suma local por tarea
`Exchange hashpartitioning(org_id, event_date, service, 8)`	partitioner y shuffle sobre la clave compuesta. `event_date` es la columna de Silver; el alias `usage_date` se aplica recién en el `HashAggregate` final (`event_date AS usage_date`), así que es la misma fecha con el nombre del grano del mart
`HashAggregate` con `sum` y `count`	reduce
`AdaptiveSparkPlan`	AQE ajusta las particiones del shuffle después de medirlas

El hash cubre la clave completa, como en §7.3, y gracias al combiner el shuffle mueve una fila por clave y tarea, no un evento.

3.4 Particiones y archivos

Tabla	Partición	Particiones	Archivos	Tamaño medio
`bronze/usage_events`	`ingest_date`	1	12	148 KB
`silver/usage_events`	`event_date`	60	60	46 KB
`gold/org_daily_usage_by_service`	`usage_date`	60	60	10 KB
`gold/cost_anomaly_mart`	`usage_date`	60	60	10 KB

El inventario completo, con filas y bytes por tabla, está en `lake_inventory.md`.

4. Calidad y Quarantine

Regla	Dónde	Acción	Datos reales	Fixtures
la línea no parsea (JSON o CSV)	Bronze	Quarantine	0	3
`event_id` nulo	Silver	Quarantine	0	1
`event_id` duplicado	stream y Silver	dedupe	0	1
`timestamp` nulo, `metric` desconocido, `unit` contra `metric`, `value` que no castea	Silver	Quarantine	0	4
`org_id` o `resource_id` sin match, recurso de otra organización	Silver	Quarantine	0	3
`cost_usd_increment` < -0,01	Silver	`cost_anomaly_flag`	211	1
`unit` nulo con `metric` conocido	Silver	imputar, `unit_imputed`	2.075	1
ticket resuelto antes de creado	Silver	Quarantine	0	1
CSAT, NPS, tasa USD, créditos, subtotal, timestamps de users, conversión sin clic	Silver	anular, reparar o marcar (D7)	40, 1, 160, 137, 13, 232 y 249, 96	1 cada una

Las tres reglas de la consigna se cumplen en Silver: 0 `event_id` nulos y 0 duplicados, 211 costos negativos marcados, y los 42.323 eventos con `value` tienen `unit`, o sea el 100% después de la imputación. Quarantine queda en 0% de los eventos y 0% del costo, contra el 1% de O3. Los datos reales no rechazan nada, así que las muestras salen de los fixtures de `tests/fixtures/landing/`, que ejercitan cada regla con un lake aparte (D10). Tres filas de Quarantine, recortadas:

Tabla	rule_name	source_file	raw_record
quarantine/usage_events_parse	parse_type_mismatch	events_fixture_0000.jsonl	{"event_id": "evt_fixture_ts", "timestamp": "not-a-timestamp", ...}, la línea cruda
quarantine/usage_events_rules	value_not_castable	events_fixture_0000.jsonl	{"event_id":"evt_fixture_value", ..., "metric":"cpu_hours", "value":"n/a", "unit":"hours", ...}, la fila de Bronze
quarantine/masters_rules	resolved_before_created	support_tickets.csv	{"ticket_id":"tkt_fixture02", ..., "created_at":"2025-07-05", "resolved_at":"2025-07-02", ...}

Una muestra completa de cada regla, con su `raw_record`, está en `quarantine_samples.md`.

5. Serving en Cassandra y AstraDB

Cassandra 5.0 corre en Docker (`infra/docker-compose.yml`). El keyspace y las tablas están en `cql/`, y las consultas en `cql/q*.cql`. AstraDB es el segundo destino, con el mismo esquema y el mismo cargador.

Consulta	Tabla	Partition key	Clustering	Lectura
Q1, costo y requests diarios por servicio en un rango	org_daily_usage_by_service	org_id	usage_date DESC, service	un rango dentro de una partición
Q2, top-N de servicios por costo en 14 días	org_daily_usage_by_service	org_id	usage_date DESC, service	14 días de una partición (84 filas como máximo), suma y orden en el cliente
Q6, días anómalos	cost_anomalies_by_org	org_id	usage_date DESC, service	un rango de una partición, filtro is_anomaly en el cliente

Replicación local: `NetworkTopologyStrategy` con `dc1: 1`. En producción sería `dc1: 3` con `LOCAL_QUORUM` para leer y escribir. Una partición tiene como máximo 6 filas por día, así que crece a unas 2.190 filas por año sin importar el tamaño de la organización, y no hace falta un bucket por mes (D11). La carga verifica, organización por organización, filas y costo contra Gold. Las 80 organizaciones y las 11.050 filas coinciden en las dos tablas.

Q1 para `org_okep7y6w` del 1 al 7 de agosto, como está en `cql/q1_daily_cost_by_service.cql` y como lo devuelve cqlsh (primeras filas de 25, `serving_q1_cqlsh.txt`):

```
SELECT usage_date, service, daily_cost_usd, requests, events
FROM cloud_analytics.org_daily_usage_by_service
WHERE org_id = 'org_okep7y6w'
  AND usage_date >= '2025-08-01' AND usage_date <= '2025-08-07';

 usage_date | service    | daily_cost_usd | requests | events
------------+------------+----------------+----------+--------
 2025-08-07 |    compute |       191.8968 |      212 |      3
 2025-08-07 | networking |         3.8985 |      378 |      6
 2025-08-06 | networking |         0.3809 |     null |      3
...
(25 rows)
```

Q2 es la misma lectura con `usage_date` del 18 al 31 de agosto: 54 filas de la partición que el cliente suma y ordena (`serving_q2_driver.txt`):

```
rank | service  | cost_14d_usd | days_with_usage
-----+----------+--------------+----------------
1    | genai    | 211.0267     | 10
2    | storage  | 117.8743     | 13
3    | database | 90.8801      | 9
```

El `null` de `requests` es un día sin requests medidos, no un cero (D6). Las salidas completas, por cqlsh y por el driver, están en `serving_q1_*`, `serving_q2_*` y `serving_q6_*`.

AstraDB se probó con la misma carga, cambiando solo la conexión: `SERVING_TARGET=astra`, el secure connect bundle por ruta, el usuario `token` y el token como contraseña, todo por variables de entorno y fuera del repo. El keyspace `cpa` se creó desde la consola, porque AstraDB no admite `CREATE KEYSPACE`, y las tablas salen del mismo `cql/01_tables.cql`. Cargamos Gold dos veces:

Tabla en AstraDB	Primera carga	Segunda carga	Gold
`cpa.org_daily_usage_by_service`	11.050	11.050	11.050
`cpa.cost_anomalies_by_org`	11.050	11.050	11.050

Q1, Q2 y Q6 devuelven sobre AstraDB los mismos resultados que en local (`serving_astra.md`, `serving_q*_driver_astra.txt`). En la evidencia copiada, el token, el host de la base y las rutas locales quedan enmascarados.

6. Analítica: anomalías de costo

`cost_anomaly_mart` compara el costo de cada día con los 28 días previos de la misma organización y servicio, con el z-score modificado `0,6745 × (x − mediana) / MAD`. Si MAD es 0, cae a la desviación absoluta media. Un día con menos de 14 días de historia no se puntúa. El corte es 3,5. El detalle y las alternativas están en D12.

Medida	Valor
filas puntuadas	7.043 de 11.050 (63,7%)
días marcados	180 (2,56% de los puntuados): 138 picos y 42 caídas
organizaciones con al menos un día marcado	69 de 80
caso más alto	`org_okep7y6w`, compute, 2025-08-07: 191,90 USD contra una mediana de 11,27 (score 106,5)

Para saber qué hace el método con los días que un umbral fijo llamaría extremos, el resumen incluye un chequeo de detección (`gold_marts.md`, `cost_anomaly_summary.json`):

Días	Total	Marcados	Puntuados sin marcar	Sin historia
costo diario por encima del p99 de Gold (63,91 USD)	110	39 (35,5%)	41	30
costo diario negativo	35	4 (11,4%)	15	16

Caso	Ejemplo	Por qué
caro y marcado	`org_g8sbi4q2`, database, 2025-08-05: 142,95 USD, mediana 9,22, score 28,9	salto contra su propia serie
caro sin marcar	`org_ktakpuxq`, compute, 2025-08-06: 108,76 USD, mediana 35,58, score 2,92	la serie gasta mucho siempre
negativo y marcado	`org_0lvsnujz`, analytics, 2025-08-24: -134,24 USD, mediana 15,61, score -10,6	corrección fuera de escala
negativo sin marcar	`org_dhylurtp`, compute, 2025-07-27: -9,00 USD, mediana 19,52, score -1,83	dentro de la variación de la serie

Hay tres ejemplos de cada caso en `gold_marts.md`. Que un día caro no se marque es lo esperado de un método relativo: un monto alto para una organización grande no es raro. Los costos negativos chicos ya quedan cubiertos por el `cost_anomaly_flag` de cada evento y por `negative_cost_events` en el mart, y el score marca solo los que se salen de su serie. Los días sin historia son la limitación más concreta: explican 30 de los días caros y 16 de los negativos que no se marcan.

No forzamos un modelo de Spark ML, porque la planificación pide no hacerlo cuando un método estadístico responde mejor, y acá responde mejor. La mediana y el MAD casi no se mueven con la cola larga medida en los incrementos (p99 de 16,69 contra un máximo de 317,43), y cada alerta se explica contra la historia de su propia serie. Un clustering diría a qué grupo se parece un día, no por qué es raro para esa organización. Para el final queda como deseable un pipeline de Spark ML con KMeans que segmente organizaciones por perfil de uso (D12).

7. Idempotencia

`make rerun-evidence` resetea el lake y el keyspace, corre el pipeline completo dos veces seguidas y guarda los conteos después de cada una (`counts_run_1.json`, `counts_run_2.json`, `idempotency.md`).

Mecanismo	Qué protege	Resultado de la segunda corrida
registro de ingestados	Landing a Bronze, batch	los 7 CSV salteados, Bronze sin cambios
checkpoints de las dos queries	Landing a Bronze, streaming	0 archivos nuevos, los mismos 12 commits en cada query
reconstrucción y dynamic partition overwrite	Silver y Gold	mismas filas en las 18 tablas del lake
Quarantine de reglas sobrescrita entera	Quarantine	sin copias del día anterior
upsert por primary key	Cassandra	las mismas 11.050 filas en cada tabla

8. Pruebas, configuración y errores

`make test` corre 43 pruebas con Spark local y sin Cassandra: esquemas sobre la muestra, carga batch (cuarentena de una fila mala, registro, encabezado que no coincide), streaming (separación antes del dedupe, nada perdido por el watermark, re-ejecución vacía), cada regla de calidad sobre los fixtures, Silver de punta a punta, la semántica de nulos de Gold y su conciliación, las anomalías (MAD, caída a la desviación media, historia mínima, ventana que solo mira hacia atrás) y el parseo de los CQL, la verificación de Java y la sal obligatoria del hash de emails.

La configuración vive en `config/pipeline.env`, copiado de `config/pipeline.env.example`, y cada variable se puede pisar desde el entorno. El pipeline acepta Java 17, 21 o 25; la evidencia se generó con 21, y con 25 pasan `make check` y las 43 pruebas. Los errores esperables (otra versión de Java, `PII_HASH_SALT` sin definir, Landing incompleto, encabezado CSV distinto del esquema, Gold que no concilia, Cassandra o AstraDB inaccesibles o sin credenciales) cortan la corrida con un mensaje en el log y código 1.

9. Gobierno y ejecución

La matriz de gobierno está en `docs/governance.md` y la ficha de ejecución en `docs/execution_sheet.md`. Los dos se incluyen al final de este documento.

10. Requisito y evidencia

Componente de §6.2	Dónde está	Evidencia
Correcciones y arquitectura	§1 y §2, `docs/architecture_v2.svg`	sin correcciones de la cátedra; diagrama v2
Batch a Bronze, 3 maestros o más	`src/pipeline/bronze_batch.py`	los 7 CSV en `lake_inventory.md`, `run_1_*.log`
Streaming a Bronze	`src/pipeline/bronze_stream.py`	`stream_bronze.md`, `run_1_stream_progress.jsonl`, `stream_state_demo_progress.jsonl`
Silver, eventos y un maestro, joins, 3 features	`src/pipeline/silver.py`	`silver_quality.md`, `data_dictionary.md`
Calidad, 3 reglas y Quarantine con muestras	`src/pipeline/quality_rules.py`, `tests/fixtures/landing/`	`silver_quality.md`, `quarantine_samples.md`
Gold `org_daily_usage_by_service`	`src/pipeline/gold.py`	`gold_marts.md`, `gold_reconciliation.json`, `gold_explain.txt`
Serving, keyspace, tabla, carga y dos consultas	`cql/`, `src/pipeline/serving.py`, `src/pipeline/queries.py`	`serving_q1_*.txt`, `serving_q2_*.txt`, `serving_astra.md`
Analítica	`src/pipeline/anomalies.py`	`gold_marts.md`, `cost_anomaly_summary.json`, `serving_q6_*.txt`
Idempotencia	`make rerun-evidence`	`idempotency.md`, `counts_run_*.json`
Gobierno preliminar	`docs/governance.md`, D17	`data_dictionary.md`, logs con secretos enmascarados
Pruebas y reproducibilidad	`tests/`, `Makefile`, `README.md`	`make test`, `run_*.json`
Backlog final	§11	este documento

El checklist §9.2 y los criterios §6.4, línea por línea, están en `evidence/delivery_2_checklist.md`.

11. Backlog para el MVP final

Prioridad	Trabajo
Obligatorio	marts `revenue_by_org_month`, `tickets_by_org_date` y `genai_tokens_by_org_date` en Gold, con sus tablas en Cassandra y las consultas Q3, Q4 y Q5
Obligatorio	confirmar con la cátedra si Q3 a Q5 son por organización o globales (§8.1 del diseño)
Obligatorio	cargar en AstraDB los marts nuevos del final, con sus consultas
Obligatorio	presentación ejecutiva, video y guion de la defensa
Deseable	vista intradía provisoria del speed layer con `foreachBatch` (D3, apéndice C)
Deseable	job de retención para Bronze (90 días) y Quarantine (180 días)
Deseable	Silver incremental por las fechas de evento que tocó cada corrida, en lugar de reconstruir todo
Deseable	autenticación y roles en Cassandra, y alertas sobre el resumen de cada corrida
Deseable	pipeline de Spark ML con KMeans para segmentar organizaciones por perfil de uso (requests, cómputo, almacenamiento, GenAI, costo), con su métrica de silueta
Fuera de alcance	Kafka u otro broker, Delta o Iceberg, Declarative Pipelines, Real-Time Mode, `transformWithState`

Gobierno preliminar

Controles iniciales de la entrega 2: quién es dueño de cada activo, qué tan sensible es, qué reglas lo protegen, cómo se rastrea y quién lo puede leer. Cada fila separa lo que el pipeline ya hace de lo que queda para el MVP final. Los dueños son los roles de `docs/design.md` §8.4.

Clasificación

Nivel	Qué entra	Ejemplos
Personal	datos que identifican a una persona	`users.email`
Confidencial	datos comerciales de un cliente	facturación, nombre de la organización, tickets, uso y costo por organización
Interna	agregados y metadatos operativos	marts sin datos personales, logs de corrida, registro de ingestados

Matriz de activos

Activo	Dueño	Sensibilidad	Acceso	Retención
Landing, 7 CSV y 120 JSONL	Ingesta	Personal (`users.csv`)	solo los jobs de ingesta y la exploración de solo lectura	histórico completo, inmutable (D8)
`bronze/usage_events`	Ingesta	Confidencial	jobs de Silver	90 días sobre `ingest_date`
`bronze/<maestro>`, 7 tablas	Ingesta	Personal (`users`), Confidencial (resto)	jobs de Silver	90 días sobre `ingest_date`
`silver/usage_events`	Calidad y Silver	Confidencial	jobs de Gold, notebooks del equipo	histórico completo
`silver/users`	Calidad y Silver	Confidencial: sin email, con `email_hash` y `email_domain`	jobs de Gold	histórico completo
`silver/billing_monthly` y demás maestros	Calidad y Silver	Confidencial	jobs de Gold	histórico completo
`gold/org_daily_usage_by_service`	Marts	Confidencial	carga a Cassandra, BI	13 meses sobre `usage_date`
`gold/cost_anomaly_mart`	Marts	Confidencial	carga a Cassandra, BI	13 meses sobre `usage_date`
`quarantine/*`, 4 tablas	Calidad y Silver	la del origen: `raw_record` puede traer un email	ingenieros del equipo, como cola de revisión	180 días las de append; las de reglas se reconstruyen
Cassandra `cloud_analytics` y AstraDB `cpa`, 2 tablas cada una	Serving	Confidencial, sin datos personales	FinOps y Producto, solo lectura; en AstraDB, por token de aplicación	la del mart de Gold
`_registry`, `_checkpoints`, `logs/`	Ingesta	Interna	ingenieros del equipo	lo que dure la query; logs 90 días

Activo	Reglas de calidad	Métricas	Linaje	Monitoreo	Evidencia
Landing	ninguna, se observa (design §5)	archivos y filas por fuente	nombre de archivo	`make check` falla si falta una fuente	`landing_profile.md`
Bronze	parsea con el esquema declarado, encabezado CSV igual al esquema	filas, filas a Quarantine, claves repetidas, progreso por micro-batch	`source_file`, `ingest_ts`, `ingest_date`, sha256 en el registro	resumen de corrida, `numRowsDroppedByWatermark`	`stream_bronze.md`, `run_*_stream_progress.jsonl`
Silver	D6 y D7: 8 reglas de rechazo de eventos, 1 de tickets, 8 flags de reparación	O3 en filas y en costo, `unit` con `value` al 100%	lo de Bronze, más `processed_ts` y `run_id`	warning si Quarantine supera el 1% (O3)	`silver_quality.md`
Gold	claves no nulas, filas igual al grano, costo a 0,01 USD de Silver por organización y día	resultado de la conciliación	`run_id`, `computed_ts`, `events` por fila	la corrida falla si no concilia y no publica	`gold_marts.md`, `gold_reconciliation.json`
Quarantine	es el control	filas por regla y por tabla	`raw_record`, `source_file`, `rule_name`, `run_id`	tamaño contra O3	`quarantine_samples.md`
Cassandra	filas y costo por organización iguales a Gold	filas por tabla	`run_id`, `computed_ts`	la carga falla si no coincide	`serving_q*.txt`, `idempotency.md`

Linaje de punta a punta

Un número servido se rastrea hasta su archivo de origen con las columnas técnicas:

1. Una fila de Cassandra trae `run_id` y `computed_ts`. Con el `run_id` se abre `logs/<run_id>.json`, que tiene la configuración, el commit de git y las métricas de cada etapa de esa corrida.
2. La misma clave (`org_id`, `usage_date`, `service`) en `gold/org_daily_usage_by_service` dice sobre cuántos eventos se apoya (`events`), cuántos son correcciones negativas y cuántos tienen `unit` imputado.
3. En `silver/usage_events`, filtrando por `org_id`, `event_date` y `service`, aparecen esos eventos con su `source_file` e `ingest_ts`.
4. El registro de ingestados y el checkpoint del stream dicen cuándo entró cada archivo de Landing, y el sha256 del registro prueba que el archivo de los maestros no cambió desde entonces.

Datos personales

`users.email` es el único dato personal del dataset. Hasta Bronze se guarda tal cual, porque Bronze es la copia fiel de la fuente. Silver lo reemplaza por `email_hash`, un sha256 de una sal más el email normalizado, y `email_domain`. El hash sirve para contar y deduplicar usuarios sin exponer la dirección. La sal (`PII_HASH_SALT`) la genera `make setup` en `config/pipeline.env`, que no se commitea. Sin sal la corrida se detiene en `make check` y en Silver, antes de escribir `users`. Ningún mart ni tabla de Cassandra tiene datos personales.

Credenciales y configuración

- Nada secreto vive en git. `config/pipeline.env`, `.env`, `*.pem` y los bundles `secure-connect-*.zip` están en `.gitignore`.
- El token y el bundle de AstraDB se leen solo de variables de entorno. El log de cada corrida muestra la configuración con los secretos reemplazados por `<set>` o `<unset>`.
- La evidencia copiada de los logs reemplaza la ruta local del repo por `<repo>`.

Lo implementado y lo pendiente

Control	Hoy	Para el MVP final
Acceso por zona	un solo usuario en la máquina local, separación por carpeta y por escritor (D15)	un rol por job con permisos por prefijo en el object storage
Acceso a Cassandra	contenedor local sin autenticación, solo en `127.0.0.1`	`PasswordAuthenticator`, un rol de carga con `MODIFY` y uno de lectura con `SELECT` sobre el keyspace
Retención	definida por zona, no automatizada	un job que borre particiones de Bronze y Quarantine vencidas por fecha de partición
Monitoreo	resumen JSON por corrida, warnings de O3, compuertas que cortan la corrida	alertas sobre el resumen y un tablero de tamaño de Quarantine
Catálogo	diccionario generado desde los Parquet (`data_dictionary.md`)	descripciones de negocio por columna, versionado de esquemas

Ficha de ejecución

Cómo corre el pipeline hoy, con los números del event log de Spark, y cómo se traduciría a containers de YARN a la escala proyectada del apéndice A del diseño. El detalle por corrida está en `evidence/entrega-2/execution_profile.md`.

Hoy: un proceso local

Recurso	Valor	Por qué
Master	`local[4]`	un JVM con driver y executor juntos y 4 slots de tarea
Memoria	`spark.driver.memory=2g`	el dataset completo son 13 MB de JSON y 7 MB de Parquet
`spark.sql.shuffle.partitions`	8	con 200 (el valor por defecto) cada agregación lanza 200 tareas casi vacías. 8 son dos rondas de 4 slots, y AQE junta las que quedan chicas
AQE	activado	coalesce de particiones de shuffle después de medir el tamaño real
Archivos por partición	1 (`FILES_PER_PARTITION`)	un archivo de 10 a 150 KB por día o por micro-batch (design §5.1)
Java	JDK 21 (`PIPELINE_JAVA_HOME`)	Spark 4.2 corre sobre 17, 21 y 25; el pipeline acepta los tres, y la evidencia se generó con 21

Jobs y lo que midió la primera corrida

Primera corrida de `make rerun-evidence`, sobre un lake vacío. Una etapa del pipeline lanza varios jobs de Spark: cada `count`, cada escritura y cada lectura de métricas es un job.

Etapa	Qué hace	Trigger	Segundos	Jobs	Tareas	Shuffle escrito
`bronze-batch`	7 CSV a Bronze, registro de ingestados	por corrida	9,0	64	64	51 KB
`bronze-stream`	120 JSONL a Bronze y Quarantine, 2 queries	`availableNow`, 10 archivos por micro-batch	6,4	24	108	4.570 KB
`silver`	7 maestros y eventos conformados, reglas, joins, dedupe	por corrida	8,1	63	141	6.176 KB
`gold`	`org_daily_usage_by_service`, conciliación	por corrida	2,1	20	126	963 KB
`anomalies`	`cost_anomaly_mart`	por corrida	2,8	21	166	1.202 KB
`serving`	upsert de los dos marts en Cassandra y verificación	por corrida	4,4	12	148	1.056 KB

La corrida completa tarda unos 33 segundos y la re-ejecución unos 24 (los tiempos varían entre corridas y con la carga de la máquina): el stream no lee nada y la carga batch saltea los 7 CSV registrados. Silver, Gold y la carga se rehacen enteros en las dos (D8).

El shuffle más grande es el de Silver: el dedupe por `event_id` es una ventana particionada por `event_id` y mueve todos los eventos. La agregación de Gold mueve menos porque agrega parcial antes del exchange, que es el combiner del §7 del diseño (`gold_explain.txt`, nodos 2 a 4).

A escala: cómo quedaría en YARN

Supuestos del apéndice A: unos 43.200 millones de eventos por día, 500.000 por segundo, 13 TB de JSON y 1,6 TB de Parquet por día.

Job	Cola	Executors	Container por executor	Paralelismo
Streaming a Bronze	`streaming`, 30% de capacidad mínima, aplicación de larga duración	10 executors × 4 cores, sin dynamic allocation	8 GB + 10% de overhead, redondeado a 9 GB y 4 vcores	unos 150 MB/s de JSON, 9 GB por micro-batch de un minuto
Silver diario	`batch`, 70%, elástica hasta 100% de noche	dynamic allocation hasta 100 × 5 cores	16 GB + overhead, 18 GB y 5 vcores	12.500 splits de 128 MB, unas 25 rondas de 500 tareas; `shuffle.partitions` cerca de 8.000 para unos 200 MB por partición en el dedupe
Gold diario	`batch`	los del job de Silver	igual	300.000 claves por día después del combiner; 200 particiones de shuffle alcanzan
Carga a Cassandra	`batch`	10 × 2 cores	4 GB	300.000 upserts por día, acotado por el cluster de Cassandra y no por Spark

En modo cluster, cada aplicación pide además un ApplicationMaster que hospeda al driver: unos 4 GB y 1 vcore. YARN asigna containers en múltiplos de `yarn.scheduler.minimum-allocation-mb`, por eso cada executor pide su memoria más el overhead redondeado hacia arriba. Con el Capacity Scheduler, la cola `streaming` tiene una capacidad mínima reservada para que el batch nocturno nunca le saque recursos al stream, y la cola `batch` puede usar lo que el stream no ocupa entre las 00:00 y las 06:00 UTC, que es la ventana de O1.

El estado del dedupe es lo que más cambia con la escala. Hoy son 43.200 claves. A 500.000 eventos por segundo, una hora de watermark son unos 1.800 millones de `event_id`, del orden de 180 GB. Eso pide el state store de RocksDB (`spark.sql.streaming.stateStore.providerClass`), que guarda el estado en el disco local de cada executor, y un watermark más corto, de unos 10 minutos, que sigue cubriendo el horizonte de un reintento. La deduplicación exacta queda en Silver, donde no tiene costo de estado.

Decisiones

Decisiones de diseño, con las alternativas que miramos y lo que cuesta cada una. D1 a D10 vienen de la entrega 1. D11 y D12 quedaron abiertas en la entrega 1 y se cierran con los datos de Gold. D13 a D17 son las que trajo la implementación. Fecha: 2026-11-08.

Las mediciones salen de `evidence/landing_profile.md`. Documento de diseño: `docs/design.md`.

#	Decisión	Estado
D1	Un único esquema explícito que cubre las dos versiones de evento	Aceptada
D2	Leer `value` como string en Bronze y castear en Silver con `try_cast`	Aceptada
D3	Híbrido estilo Lambda, con un speed layer sin ventanas (diferido)	Aceptada
D4	Cinco zonas, Parquet con Snappy	Aceptada
D5	Una sola columna de fecha, `ingest_date` en Bronze y `event_date` en Silver	Aceptada
D6	Política de calidad: primero imputar	Aceptada
D7	Acción por fuente, y dos supuestos de escala	Aceptada
D8	Landing inmutable, idempotencia por sobreescritura y upsert	Aceptada
D9	Convertir moneda por factura, y forzar USD a 1,0	Aceptada
D10	Las muestras de quarantine salen de fixtures en la entrega 2	Aceptada
D11	Modelo query-first en Cassandra: una partición por organización, días como clustering	Aceptada
D12	Anomalías de costo: z-score modificado con MAD sobre una ventana de 28 días, corte 3,5	Aceptada
D13	Carga batch: registro de ingestados, y el dedupe por clave va en Silver	Aceptada
D14	Streaming: `availableNow` para corridas y demo, una query y un checkpoint por destino	Aceptada
D15	Quarantine en cuatro tablas, una por escritor	Aceptada
D16	Carga a Cassandra con el driver de Python desde `foreachPartition`, local y AstraDB	Aceptada
D17	`users.email` se reemplaza en Silver por un hash con sal y el dominio	Aceptada

Cada registro, con su contexto, las alternativas que miramos y las consecuencias, está en `DECISIONS.md` en el repositorio. Acá va solo el índice, porque el cuerpo del diseño ya argumenta cada decisión donde se usa.